REPORT

On

Micro Project - Crytography and Network Security

Submitted in Partial Fulfillment of Award of

BACHELOR OF TECHNOLOGY

In

Computer Science and Engineering

By

Jenifer Christina-2411021061337
Devashree BS-2411021061319
Asfina Magi-2411021061405
Sinchana KA-2411021061296

Under theSupervision of

Mr.Sreekumar N R

ALLIANCE COLLEGE OF ADVANCED COMPUTING
ALLIANCE UNIVERSITY
BENGALURU
Oct 2

1. Problem Statement

In today's digital world, files are constantly created, downloaded, transferred, shared, and stored across many devices and systems. Somewhere along the way, a file may get changed by accident or modified on purpose without the user ever knowing. Such changes can affect the correctness, reliability, and security of the information stored in the file.

For example, think of an important document, software file, configuration file, database backup, or digital evidence file. After the original is created, it may be sent to another computer or kept in storage for a long time. In such cases, we need a way to confirm that the file is still exactly the same as the original. Checking only the file name, extension, or file size is not enough, because these can stay the same even when the actual contents have been changed.

A cryptographic hash offers an efficient solution to this problem. A hash function takes the contents of a file as input and produces a fixed-length value known as a hash value or digest. We can think of it as a digital fingerprint of the file: if the contents change, the hash changes too.

This micro project focuses on building a simple **File Integrity Verification application using the SHA-256 cryptographic hashing algorithm**. The application generates a SHA-256 hash for a selected file and stores it as a reference value. When the file is checked again, it calculates the hash once more and compares it with the stored reference hash.

If the two hash values are identical, the application reports that the file has not been modified. If they differ, it reports that the file has been modified.

Through this project, we gained a practical understanding of cryptographic hashing and saw how SHA-256 can be applied to a real-world security problem. It also helped us appreciate why file integrity matters in areas such as digital forensics, cybersecurity, software verification, and security monitoring.

2. Methodology

The proposed system uses **SHA-256 cryptographic hashing** to create a digital fingerprint of a file and find out whether its contents have been changed. The approach is simple: generate a reference hash first, then compare it with a freshly generated hash during verification.

2.1 Flow of Work

Start → Select File → Read File → Generate SHA-256 Hash → Store Reference Hash → Verify File → Generate Current Hash → Compare Hashes → Display Result → End

2.2 Working Methodology

1. File Selection:
The user selects the file to be verified through the application.

2. File Reading:
The selected file is opened in binary mode, so different file types such as text files, images, and PDFs can all be processed.

3. Hash Generation:
The file contents are processed using SHA-256 to produce a 256-bit hash, shown as a 64-character hexadecimal value. This hash works as the file's digital fingerprint.

4. Reference Hash:
The hash generated from the original file is saved as the reference hash, which represents the expected state of the file.

5. Verification:
When the file is checked again, a new SHA-256 hash is generated from its current contents.

6. Hash Comparison:
The current hash is compared with the stored reference hash.

Reference Hash = Current Hash → File Integrity Verified

Reference Hash ≠ Current Hash → File Modification Detected

7. Result Display:
Based on the comparison, the application shows the user the appropriate result.

2.3 Properties of SHA-256

SHA-256 is deterministic, which means the same input always gives the same hash. It produces a fixed 256-bit output no matter how large the file is, and it is very sensitive to changes, so even a tiny modification can give a completely different hash. It is also a one-way function, which makes it extremely hard to recover the original data from the hash.

2.4 Practical Example

For example, if the original file contains “This is my original file.”, its SHA-256 hash is stored as the reference. If the file stays unchanged, the same hash is generated and the integrity is verified. If the content is edited, a different hash is produced and the application reports File Modification Detected.

In short, this methodology gives a simple and effective way to detect accidental or unauthorized changes to digital files.

3. Tools and Technologies Used

We developed this project using Python and its standard libraries for file handling and cryptographic hashing. The tools we chose are simple, lightweight, and well suited for file integrity verification.

3.1 Technologies and Tools

Tool / Technology	**Purpose**
Python	Main programming language
hashlib	Generates SHA-256 hash values
File Handling	Reads files in binary mode
Visual Studio Code	Development and testing
Windows OS	Development and testing environment
SHA-256	Cryptographic algorithm used for integrity verification

3.2 Python

We used Python as the main programming language because of its simple syntax and built-in support for file handling and cryptographic operations.

3.3 hashlib

We used Python's hashlib library to generate the SHA-256 hash of the selected file. It offers a reliable, standard implementation of hashing algorithms, so we did not have to write the algorithm ourselves.

3.4 File Handling

Files are read in binary mode so that the application can handle different file types such as text files, PDFs, images, and documents.

3.5 Visual Studio Code

We used Visual Studio Code to write, run, debug, and test the Python application.

3.6 SHA-256

SHA-256 is the core algorithm of this project. It converts file contents into a fixed 256-bit hash value, which is then used as a reference for later integrity verification**.**

4. Results and Analysis

We tested the application using a sample file to check whether SHA-256 could correctly identify changes in file contents. Multiple test cases were carried out to evaluate both successful verification and modification detection.

4.1 Test Case 1 — Generating the Reference Hash

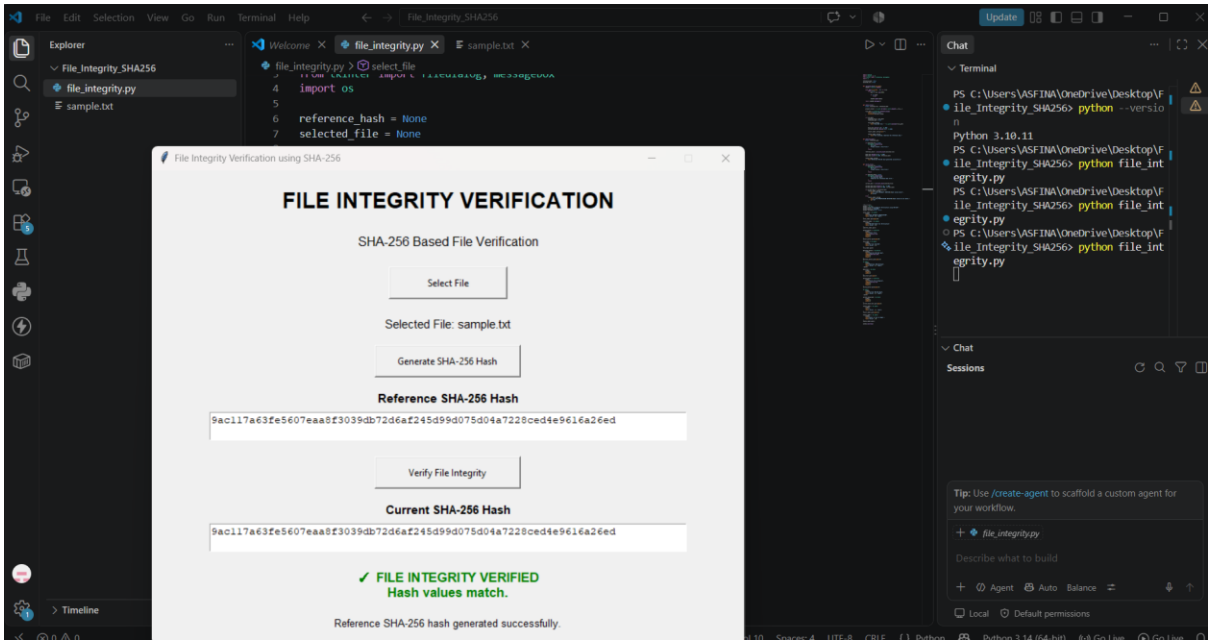
We created a sample file named **sample.txt** for testing. It contained simple text and was selected through the application.

The application read the file contents and generated its SHA-256 hash, which was then stored as the reference value.

Expected Result:
A valid SHA-256 hash should be generated and stored successfully.

Observed Result:
The application generated a 64-character hexadecimal SHA-256 hash for the selected file.

Figure 1: SHA-256 hash generated for the original file



This hash represents the state of the file at the time of the initial check. It can later be used to find out whether the file contents have changed.

4.2 Test Case 2 — Verification of an Unmodified File

Next, we selected the same file again without making any changes to its contents.

The application calculated the SHA-256 hash again and compared the new value with the stored reference value.

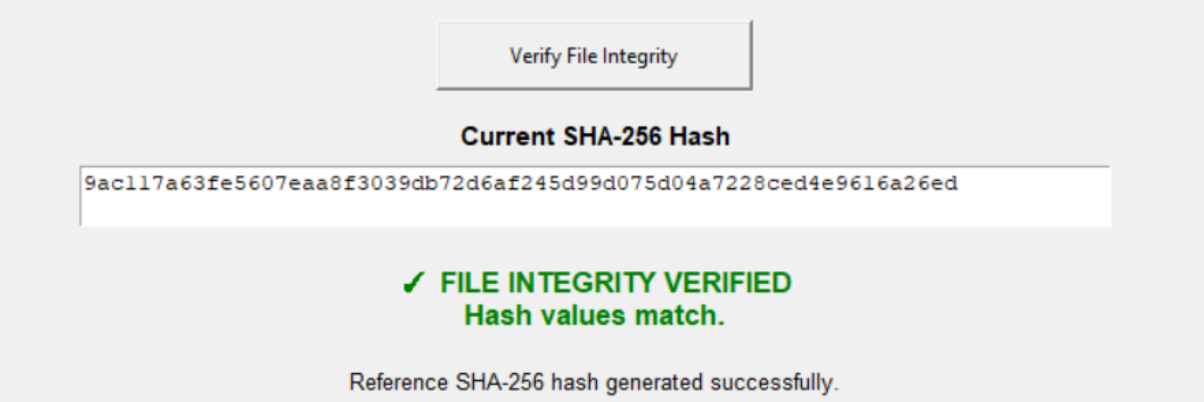
Expected Result:

The two hash values should be identical because the file contents have not changed.

Observed Result:

The reference hash and current hash were identical, so the application reported that the file integrity was verified.

Figure 2: Successful verification of an unchanged file



Result

Reference Hash = Current Hash

Therefore:

File Integrity Verified

This test confirms that the application correctly recognizes an unchanged file.

4.3 Test Case 3 — Verification After File Modification

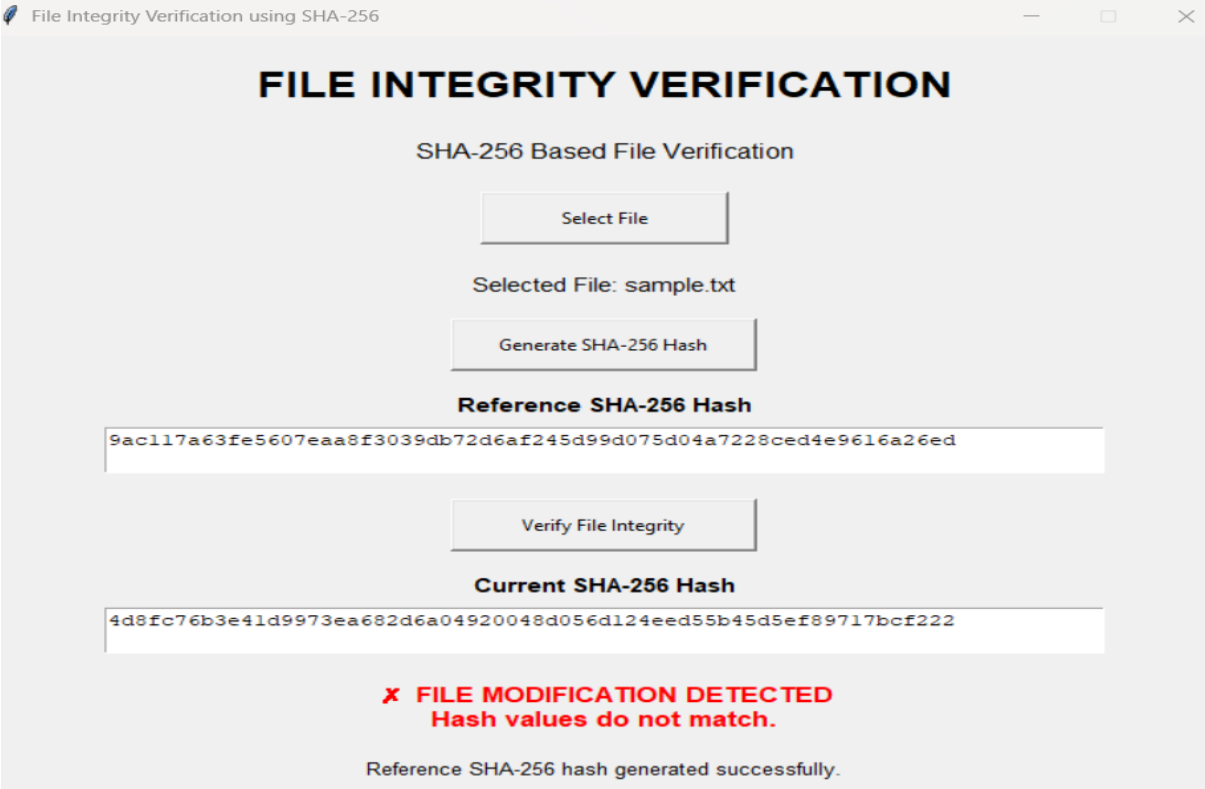
To test the modification detection, we intentionally changed the contents of **sample.txt** after the reference SHA-256 hash had been generated.

We made a small change to the file, while the stored reference hash was kept as it was. The modified file was then verified using the application.

Expected Result:
The current SHA-256 hash should be different from the stored reference hash, and the application should report that the file has been modified.

Observed Result:
The application generated a new SHA-256 hash that was different from the original reference hash, detected the modification, and displayed a **"FILE MODIFICATION DETECTED"** message.

Figure 3: Modified file detected through SHA-256 hash comparison



Result

Reference Hash ≠ Current Hash

Therefore:

File Modification Detected

The result shows that even a small change in the file contents produces a different SHA-256 hash. This lets the application identify whether the file was altered after the reference hash was generated.

4.4 Observation

This experiment shows the deterministic property of SHA-256. When exactly the same file contents are given as input, the algorithm produces the same hash value, and when the contents change, a different hash value is produced.

The three test cases together show that the application can **generate a reference hash, verify an unchanged file, and detect modifications**.

5. Advantages

1. **Simple and easy to use:** The application gives a straightforward way to check whether a file has been modified.
2. **Fast verification:** Since SHA-256 produces a fixed-length hash, hash values can be compared quickly.
3. **Detects small changes:** Even a tiny change in the file contents results in a different hash value.
4. **Works with different file types:** Because files are processed as binary data, it works with text files, documents, images, and other formats.
5. **Useful for security monitoring:** Hash comparison helps spot unexpected changes to important files.
6. **Relevant to digital forensics:** Hash values can be used to show that a digital file has stayed unchanged during examination or handling.
7. **Uses a standard cryptographic algorithm:** SHA-256 is a widely used cryptographic hash algorithm and is available in Python's standard library.

6. Limitations

Although the application is effective at detecting file modifications, it has a few limitations.

1. **Reference hash must be protected:** If an attacker can modify both the file and its stored reference hash, the modification may go undetected.
2. **Hashing does not prevent modification:** SHA-256 only helps us detect changes; it cannot stop someone from modifying the file.
3. **No authentication mechanism:** The current application does not verify who created or stored the reference hash.
4. **Single-file verification:** The basic version is designed mainly for checking individual files rather than monitoring an entire system.
5. **No automatic continuous monitoring:** The user has to start the verification manually; the application does not monitor files continuously in the background.

7. Conclusion and Future Work

7.1 Conclusion

The **File Integrity Verification using SHA-256** project showed us how cryptographic hashing can be used to detect changes in digital files.

The application generates a SHA-256 hash for a selected file and stores it as a reference value. During verification, a new hash is generated and compared with the reference. If both values match, the file is considered unchanged; if they differ, a modification is detected.

Testing with both an unchanged and a modified file gave the expected results. Through this project, we gained practical understanding of **SHA-256, file handling, Python's hashlib library, and file integrity verification**.

The project also showed us why file integrity matters in areas such as **cybersecurity, security monitoring, software verification, and digital forensics**.

7.2 Future Work

The application can be extended to support **multiple files and folders**, real-time file monitoring, and automatic alerts when changes are detected. Future versions could also include **secure hash storage, timestamps, verification history, user authentication, and digital signatures** to improve both security and usability.

These enhancements could make the application more practical for file integrity monitoring and forensic use.

8. References

1. NIST, *Secure Hash Standard (SHS)*, FIPS PUB 180-4.
2. Python Software Foundation, *hashlib — Secure hashes and message digests*, Python Documentation.
3. NIST, *Cryptographic Standards and Guidelines*.
4. William Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson.